FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	**OMBROG, Xavier Michael Emmanuel, N.**
Section:	**AN43**
Professor:	
Date:	

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.

2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

```
Random Walk Simulation (Console-Based)
```

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Output of the Provided Code (Steps 1 to 4)

I ran the provided code first using the suggested inputs. The console only asks for the three values below, then the grid window opens and animates the agents.

Console Output - Provided Code (Step 1 inputs)

```
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
```

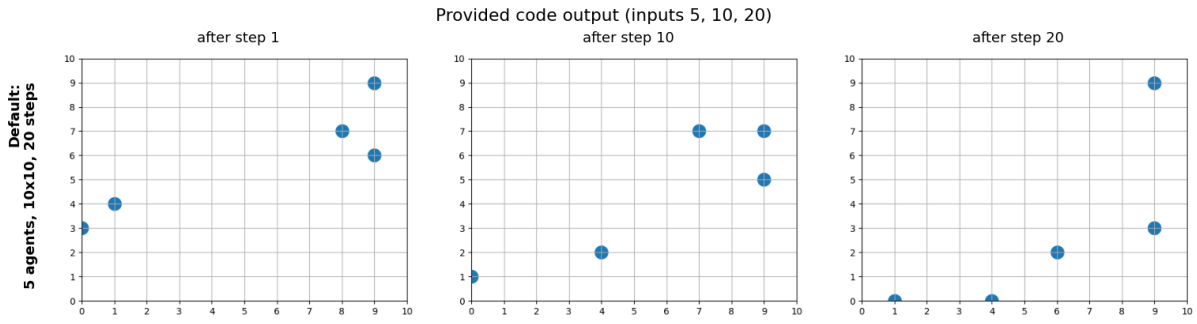


Figure 1. Provided code with 5 agents, a 10x10 grid and 20 steps, captured after steps 1, 10 and 20.

Step 3 observation. Each dot moves one cell per step, so the motion looks like jittery zig-zags rather than straight travel. Over the 20 steps the five agents drifted to new spots around the grid without forming any

lasting cluster. The rule ignores other agents, so two dots can land on the same cell and look like one. When a move would cross the edge, the agent just stays where it is for that step, which is why several dots sit on the edge rows and columns (0 or 9).

Step 4 recorded observations. I repeated the run with different values (Figure 2 shows each run after its first, middle and last step). The table below records what changed.

Experiment	Inputs (agents, grid, steps)	What changed
More agents	20, 10, 20	The grid looks much busier and agents keep landing on the same cells. Only 16 dots were visible after step 1 and 18 after step 20, because agents that share a cell are drawn on top of each other.
Smaller grid	5, 5, 20	Agents hit the walls constantly. All 5 agents were on an edge cell after step 1 and again after step 20, and the visible dot count dropped to 3 or 4 at times because they kept overlapping.
Larger grid	5, 20, 20	Agents stay in their own neighbourhoods and look almost still compared with the grid. With only 100 moves in total, at most about a quarter of the 400 cells can be reached, and only 1 or 2 agents were on an edge cell in the captured frames.
Longer run	5, 10, 50	Agents wander much farther, and the layout at step 50 looks very different from step 1. Pairs of agents sometimes end up next to each other by chance, but nothing pulls them together.

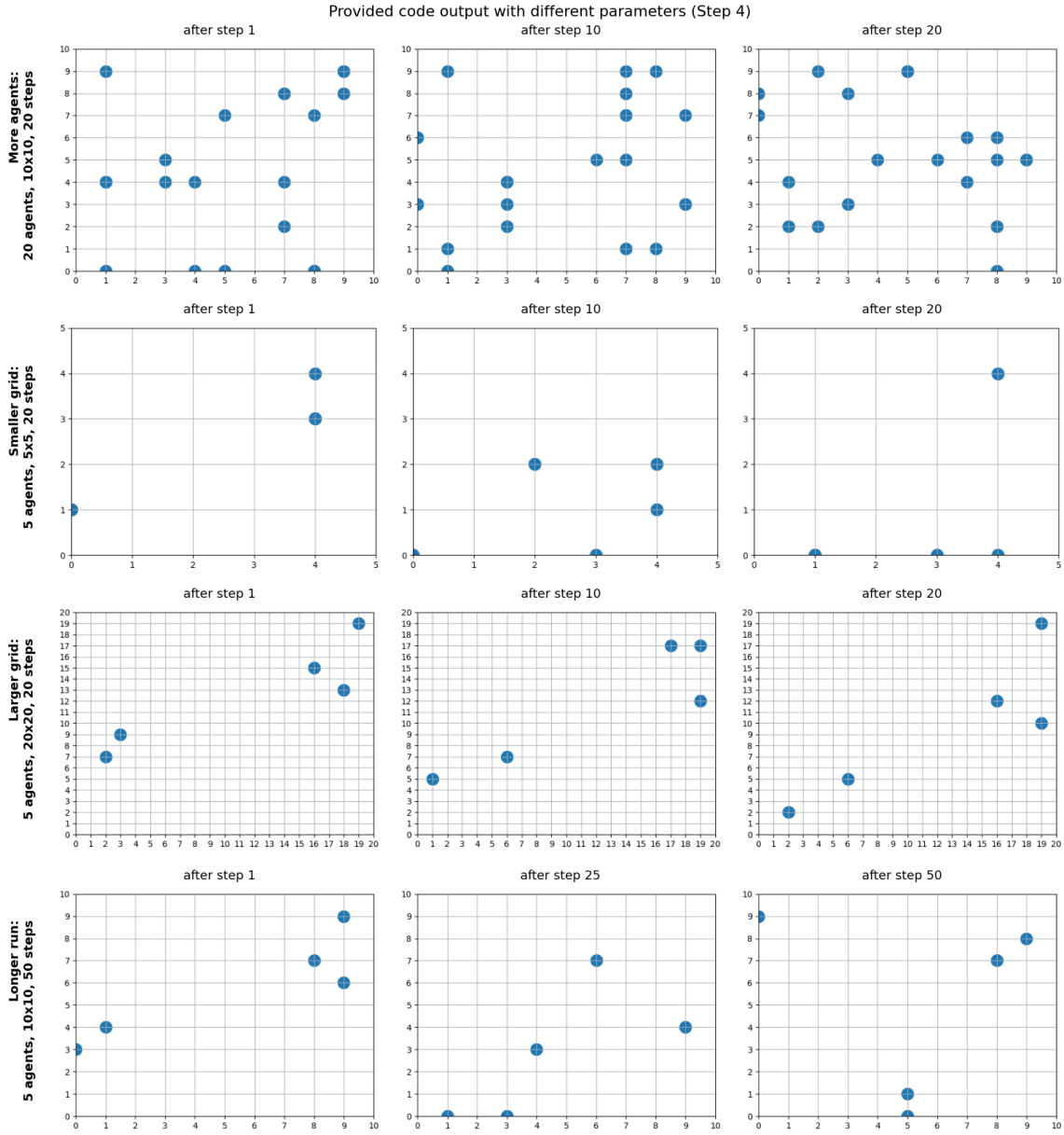


Figure 2. Step 4 experiments with the provided code, each captured after the first, middle and last step.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.



Modified (Updated Code - Laboratory Tasks)

I modified the provided code into TA1_random_walk.py. For Task 1, every agent keeps a path list and adds its cell after each step; the animation draws each agent's trail in its own color, the console prints a path summary, and the full step-by-step paths are saved to TA1_agent_paths.csv. For Task 2, the user picks one of three movement rules for each simulation: Rule 1 is the original random walk, Rule 2 makes agents avoid each other by only stepping into empty neighbouring cells, and Rule 3 adds a preferred direction that is chosen 50% of the time. For Task 3, the program prints every final position, a text map of the grid and distribution measures (centroid, standard deviation, quadrant counts, crowding and grid coverage), and saves the final positions to TA1_final_positions.csv. For Task 4, two simulations (A and B) are entered and run together, then compared in one table and in side-by-side charts.

A few other things were cleaned up. Positions now live in AgentPy's Grid (the provided code creates the grid but never moves the agents on it), the whole run is simulated first and the animation only replays it so the charts and the console always match, all inputs are validated, and an optional seed makes any run reproducible.

TA1_random_walk.py (Modified Code)

```
"""
CS0065 - Intelligent Systems
Technical Assessment 1: Random Walk Simulation Using AgentPy (Modified Code)

This program extends the provided random walk code to cover the four
laboratory tasks:

  Task 1 - Track agent paths: every agent records each cell it occupies.
           The paths are drawn as trails in the animation, printed in the
           console, and saved to TA1_agent_paths.csv.
  Task 2 - Change movement rules: the user picks one of three rules.
             [1] Random walk         - the original rule (25% per direction)
             [2] Avoid each other    - never step into a cell another agent is in
             [3] Preferred direction - biased walk toward a chosen direction
  Task 3 - Analyze final positions: final positions are printed, mapped,
           saved to TA1_final_positions.csv, and summarized (centroid,
           spread, quadrant counts, crowding, grid coverage).
  Task 4 - Compare scenarios: two simulations (A and B) are run with
           user-entered parameters and compared side by side.

Libraries needed: agentpy, matplotlib  (pip install agentpy matplotlib)
"""

import csv
import math
import random
from collections import Counter

import agentpy as ap
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.lines import Line2D
from matplotlib.ticker import MaxNLocator

# ---------------------------------------------------------------------------
# Constants
# ---------------------------------------------------------------------------
# The four possible moves (same four directions as the original code)
DIRECTIONS = {'R': (1, 0), 'L': (-1, 0), 'U': (0, 1), 'D': (0, -1)}
DIRECTION_NAMES = {'R': 'Right', 'L': 'Left', 'U': 'Up', 'D': 'Down'}
RULE_NAMES = {1: 'Random walk', 2: 'Avoid each other', 3: 'Preferred direction'}
PREFERRED_PROB = 0.50     # Rule 3: chance of stepping in the preferred direction
MAX_TABLE_ROWS = 25       # Longest per-agent table printed in the console
LINE = '=' * 78


# ---------------------------------------------------------------------------
# Console input helpers (re-ask until the input is valid)
# ---------------------------------------------------------------------------
def ask_int(prompt, minimum, maximum=None):
    """Ask for a whole number within [minimum, maximum]."""
    while True:
```

```
        text = input(prompt).strip()
        try:
            value = int(text)
        except ValueError:
            print("   Invalid input: please enter a whole number.")
            continue
        if value < minimum or (maximum is not None and value > maximum):
            if maximum is None:
                print(f"   Invalid input: the value must be at least {minimum}.")
            else:
                print(f"   Invalid input: the value must be from {minimum} to {maximum}.")
            continue
        return value


def ask_direction(prompt):
    """Ask for one of U, D, L, R (not case sensitive)."""
    while True:
        text = input(prompt).strip().upper()
        if text in DIRECTIONS:
            return text
        print("   Invalid input: please type U, D, L or R.")


def ask_seed():
    """Optional seed so that a run can be reproduced exactly."""
    text = input("Enter random seed (press Enter for a random seed): ").strip()
    if text == '':
        seed = random.randint(1, 999999)
        print(f"   Using random seed: {seed}")
        return seed
    try:
        return int(text)
    except ValueError:
        seed = random.randint(1, 999999)
        print(f"   Not a whole number, using random seed: {seed}")
        return seed


def get_scenario(label, seed):
    """Collect the parameters of one simulation from the console."""
    print(f"\n--- Simulation {label} ---")
    num_agents = ask_int("Enter number of agents: ", 1)
    grid_size = ask_int("Enter grid size (e.g., 10 for 10x10): ", 2)
    num_steps = ask_int("Enter number of steps: ", 1)
    print("Movement rules: [1] Random walk  [2] Avoid each other  [3] Preferred direction")
    rule = ask_int("Choose movement rule (1-3): ", 1, 3)
    preferred = ask_direction("Enter preferred direction (U/D/L/R): ") if rule == 3 else None
    return {
        'label': label,
        'agents': num_agents,
        'grid_size': (grid_size, grid_size),
        'steps': num_steps,          # AgentPy stops the run after this many steps
        'rule': rule,
        'preferred': preferred,
        'seed': seed,                # AgentPy seeds model.random with this value
    }


def direction_key(delta):
    """Turn a move such as (1, 0) into its letter ('R')."""
    for key, value in DIRECTIONS.items():
        if value == delta:
            return key
    return None


def describe(p):
    """One-line description of a parameter set."""
    rule = RULE_NAMES[p['rule']]
    if p['rule'] == 3:
        rule += f" ({DIRECTION_NAMES[p['preferred']]})"
    size = p['grid_size'][0]
    return f"{rule}, {p['agents']} agents, {size}x{size} grid, {p['steps']} steps"


# ---------------------------------------------------------------------------
# Agent Definition
```

```
# ---------------------------------------------------------------------------
class RandomWalker(ap.Agent):

    def setup(self):
        self.path = []                              # Task 1: every cell occupied, in order
        self.moves = 0                              # steps where the agent changed cell
        self.stayed = 0                             # steps where the agent could not move
        self.chosen_counts = Counter()              # Task 2: directions picked by the rule
        self.direction_counts = Counter()           # Task 2: directions actually moved

    @property
    def position(self):
        # The AgentPy grid is the single source of truth for every position
        return self.model.grid.positions[self]

    def step(self):
        x, y = self.position
        width, height = self.model.p.grid_size

        if self.model.p.rule == 2:
            new_pos = self.avoid_move(x, y, width, height)
        else:
            if self.model.p.rule == 3:
                dx, dy = self.biased_direction()
            else:
                dx, dy = self.model.random.choice(list(DIRECTIONS.values()))
            self.chosen_counts[direction_key((dx, dy))] += 1
            # Keep agents inside grid (same clamping as the original code)
            new_x = max(0, min(width - 1, x + dx))
            new_y = max(0, min(height - 1, y + dy))
            new_pos = (new_x, new_y)

        # Bookkeeping for the analysis
        if new_pos == (x, y):
            self.stayed += 1
        else:
            self.moves += 1
            self.direction_counts[direction_key((new_pos[0] - x, new_pos[1] - y))] += 1

        self.model.grid.move_to(self, new_pos)
        self.path.append(new_pos)                   # Task 1: record the new cell

    def biased_direction(self):
        """Rule 3: go the preferred way 50% of the time, otherwise pick one
        of the other three directions (about 16.7% each)."""
        preferred = self.model.p.preferred
        if self.model.random.random() < PREFERRED_PROB:
            return DIRECTIONS[preferred]
        others = [delta for key, delta in DIRECTIONS.items() if key != preferred]
        return self.model.random.choice(others)

    def avoid_move(self, x, y, width, height):
        """Rule 2: only consider neighbouring cells that are inside the grid
        and empty. If every neighbouring cell is taken, stay in place."""
        options = []
        for dx, dy in DIRECTIONS.values():
            nx, ny = x + dx, y + dy
            if 0 <= nx < width and 0 <= ny < height and not self.model.is_occupied((nx, ny)):
                options.append((nx, ny))
        if options:
            choice = self.model.random.choice(options)
            self.chosen_counts[direction_key((choice[0] - x, choice[1] - y))] += 1
            return choice
        return (x, y)


# ---------------------------------------------------------------------------
# Model Definition
# ---------------------------------------------------------------------------
class RandomWalkModel(ap.Model):

    def setup(self):
        width, height = self.p.grid_size

        # Create agents first so their IDs are 1, 2, 3, ...
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
```

```
        # Initialize random positions: each agent gets its own cell when there
        # is room; if there are more agents than cells, cells must be shared.
        cells = [(x, y) for x in range(width) for y in range(height)]
        if self.p.agents <= len(cells):
            starts = self.random.sample(cells, self.p.agents)
        else:
            starts = [self.random.choice(cells) for _ in range(self.p.agents)]
        self.grid.add_agents(self.agents, positions=starts)
        for agent in self.agents:
            agent.path.append(agent.position)

        # Histories used for the spread-over-time chart (filled in update)
        self.visited = set()
        self.coverage_history = []           # % of cells visited so far
        self.displacement_history = []       # average distance from start
        self.max_stack = 0                   # most agents ever sharing one cell

    def is_occupied(self, pos):
        """True if at least one agent is standing on the cell."""
        return len(self.grid.grid.agents[pos]) > 0

    def step(self):
        # Agents move one at a time in a new random order every step,
        # so no agent always gets the first pick of the free cells.
        order = list(self.agents)
        self.random.shuffle(order)
        for agent in order:
            agent.step()

    def update(self):
        # Runs after setup (t = 0) and after every step
        width, height = self.p.grid_size
        positions = [agent.position for agent in self.agents]
        self.visited.update(positions)
        self.coverage_history.append(100 * len(self.visited) / (width * height))
        distances = [math.dist(agent.path[0], agent.position) for agent in self.agents]
        self.displacement_history.append(sum(distances) / len(distances))
        self.max_stack = max(self.max_stack, max(Counter(positions).values()))


# ---------------------------------------------------------------------------
# Analysis helpers
# ---------------------------------------------------------------------------
def analyze(model):
    """Summary statistics used in Tasks 2, 3 and 4."""
    width, height = model.p.grid_size
    finals = [agent.position for agent in model.agents]
    n = len(finals)
    xs = [pos[0] for pos in finals]
    ys = [pos[1] for pos in finals]
    cx, cy = sum(xs) / n, sum(ys) / n
    sx = math.sqrt(sum((x - cx) ** 2 for x in xs) / n)
    sy = math.sqrt(sum((y - cy) ** 2 for y in ys) / n)
    center = ((width - 1) / 2, (height - 1) / 2)

    # Quadrants: x >= half_x is the right half, y >= half_y is the upper half
    half_x, half_y = math.ceil(width / 2), math.ceil(height / 2)
    quadrants = Counter()
    for x, y in finals:
        if x >= half_x and y >= half_y:
            quadrants['Q1'] += 1
        elif x < half_x and y >= half_y:
            quadrants['Q2'] += 1
        elif x < half_x:
            quadrants['Q3'] += 1
        else:
            quadrants['Q4'] += 1

    cell_counts = Counter(finals)
    all_inside = all(0 <= x < width and 0 <= y < height
                     for agent in model.agents for x, y in agent.path)
    directions, chosen = Counter(), Counter()
    for agent in model.agents:
        directions.update(agent.direction_counts)
        chosen.update(agent.chosen_counts)

    return {
        'finals': finals,
```

```
        'centroid': (cx, cy),
        'std': (sx, sy),
        'spread': sum(math.dist((cx, cy), pos) for pos in finals) / n,
        'center_dist': sum(math.dist(center, pos) for pos in finals) / n,
        'quadrants': quadrants,
        'border': sum(1 for x, y in finals if x in (0, width - 1) or y in (0, height - 1)),
        'cell_counts': cell_counts,
        'occupied': len(cell_counts),
        'max_cell': max(cell_counts.values()),
        'displacement': model.displacement_history[-1],
        'avg_moves': sum(agent.moves for agent in model.agents) / n,
        'avg_stayed': sum(agent.stayed for agent in model.agents) / n,
        'total_stayed': sum(agent.stayed for agent in model.agents),
        'coverage': model.coverage_history[-1],
        'density': n / (width * height),
        'all_inside': all_inside,
        'directions': directions,
        'chosen': chosen,
        'max_stack': model.max_stack,
    }


def fmt_pos(pos):
    return f"({pos[0]},{pos[1]})"


def print_header(title):
    print("\n" + LINE)
    print(title)
    print(LINE)


def print_paths(model, label):
    """Task 1 console output."""
    p = model.p
    print(f"\nSimulation {label}: {describe(p)}")
    print(f"{'Agent':>5}  {'Start':<8} {'Moves':>5} {'Stayed':>6} {'Visited':>7}"
          f"  Path (first 5 positions)")
    print(f"{'-' * 5}  {'-' * 8} {'-' * 5} {'-' * 6} {'-' * 7}  {'-' * 40}")
    for agent in list(model.agents)[:MAX_TABLE_ROWS]:
        preview = "->".join(fmt_pos(pos) for pos in agent.path[:5])
        if len(agent.path) > 5:
            preview += " ..."
        print(f"{agent.id:>5}  {fmt_pos(agent.path[0]):<8} {agent.moves:>5} "
              f"{agent.stayed:>6} {len(set(agent.path)):>7}  {preview}")
    hidden = len(model.agents) - MAX_TABLE_ROWS
    if hidden > 0:
        print(f"  ... {hidden} more agents (all paths are in TA1_agent_paths.csv)")
    print("  Moves = steps that changed cell, Stayed = steps spent in place,"
          " Visited = distinct cells")


def print_rule_check(model, stats, label):
    """Task 2 console output: shows that each rule behaves as designed."""
    p = model.p
    print(f"\nSimulation {label}: {describe(p)}")
    if p['rule'] == 1:
        rule_text = "picks Up/Down/Left/Right with equal chance (25% each)"
        stay_text = "edge of the grid"
    elif p['rule'] == 2:
        rule_text = "picks randomly among the empty neighbouring cells inside the grid"
        stay_text = "no empty neighbouring cell"
    else:
        rule_text = (f"moves {DIRECTION_NAMES[p['preferred']]} {PREFERRED_PROB:.0%} of the time,"
                     f" other directions {(1 - PREFERRED_PROB) / 3:.1%} each")
        stay_text = "edge of the grid"

    def shares(counts):
        total = sum(counts.values())
        if total == 0:
            return "none"
        text = " | ".join(f"{DIRECTION_NAMES[k]} {100 * counts[k] / total:.1f}%"
                          for k in ('R', 'L', 'U', 'D'))
        return f"{text}  (n = {total})"

    print(f"  Rule              : {rule_text}")
    print(f"  Directions chosen : {shares(stats['chosen'])}")
    print(f"  Moves made        : {shares(stats['directions'])}")
```

```
    print(f"  Stayed put        : {stats['total_stayed']} time(s) (blocked by: {stay_text})")
    print(f"  Most agents sharing one cell at any step: {stats['max_stack']}")


def print_final_positions(model, stats, label):
    """Task 3 console output."""
    p = model.p
    width, height = p.grid_size
    print(f"\nSimulation {label}: {describe(p)}")

    # Final positions, six per line
    print("Final positions (agent: x,y):")
    items = [f"{agent.id:>3}:{fmt_pos(agent.position):<8}" for agent in model.agents]
    for i in range(0, len(items), 6):
        print("  " + "  ".join(items[i:i + 6]))

    # Text map of the final distribution (only for grids that fit the console)
    if width <= 30:
        print("Final position map (number = agents in the cell, . = empty,"
              f" top row is y = {height - 1}):")
        for y in range(height - 1, -1, -1):
            row = ""
            for x in range(width):
                count = stats['cell_counts'].get((x, y), 0)
                row += f"{count if count else '.':>3}"
            print(f"  y={y:>2} |{row}")
        print("       +" + "---" * width)
        print("     x =" + "".join(f"{x:>3}" for x in range(width)))

    q = stats['quadrants']
    n = len(stats['finals'])
    cx, cy = stats['centroid']
    print(f"  Centroid (average x, y)          : ({cx:.2f}, {cy:.2f})")
    print(f"  Std. deviation of x, y           : ({stats['std'][0]:.2f}, {stats['std'][1]:.2f})")
    print(f"  Avg distance from the centroid   : {stats['spread']:.2f} cells")
    print(f"  Avg distance from grid center    : {stats['center_dist']:.2f} cells")
    print(f"  Quadrants  Q1 upper-right: {q['Q1']} | Q2 upper-left: {q['Q2']}"
          f" | Q3 lower-left: {q['Q3']} | Q4 lower-right: {q['Q4']}")
    print(f"  Agents on the border cells       : {stats['border']} of {n}"
          f" ({100 * stats['border'] / n:.1f}%)")
    print(f"  Occupied cells at the end        : {stats['occupied']} of {width * height}"
          f" (max {stats['max_cell']} agent(s) in one cell)")
    print(f"  Grid coverage (cells visited)    : {stats['coverage']:.1f}%")
    check = "PASSED - every recorded position is inside the grid" if stats['all_inside'] \
        else "FAILED - an agent left the grid"
    print(f"  Boundary check                   : {check}")


def print_comparison(model_a, stats_a, model_b, stats_b):
    """Task 4 console output."""
    def size(model):
        return f"{model.p.grid_size[0]}x{model.p.grid_size[1]}"

    def rule(model):
        text = RULE_NAMES[model.p.rule]
        if model.p.rule == 3:
            text = f"Preferred ({DIRECTION_NAMES[model.p.preferred]})"
        return text

    rows = [
        ("Number of agents", model_a.p.agents, model_b.p.agents),
        ("Grid size", size(model_a), size(model_b)),
        ("Number of steps", model_a.p.steps, model_b.p.steps),
        ("Movement rule", rule(model_a), rule(model_b)),
        ("Agent density (agents per cell)",
         f"{stats_a['density']:.2f}", f"{stats_b['density']:.2f}"),
        ("Avg distance from start",
         f"{stats_a['displacement']:.2f}", f"{stats_b['displacement']:.2f}"),
        ("Avg moves per agent", f"{stats_a['avg_moves']:.1f}", f"{stats_b['avg_moves']:.1f}"),
        ("Avg steps stayed in place",
         f"{stats_a['avg_stayed']:.1f}", f"{stats_b['avg_stayed']:.1f}"),
        ("Grid coverage", f"{stats_a['coverage']:.1f}%", f"{stats_b['coverage']:.1f}%"),
        ("Avg distance from centroid", f"{stats_a['spread']:.2f}", f"{stats_b['spread']:.2f}"),
        ("Agents on border cells", f"{100 * stats_a['border'] / model_a.p.agents:.1f}%",
         f"{100 * stats_b['border'] / model_b.p.agents:.1f}%"),
        ("Occupied cells at the end", stats_a['occupied'], stats_b['occupied']),
        ("Max agents in one cell (end)", stats_a['max_cell'], stats_b['max_cell']),
    ]
```

```
    print(f"{'Metric':<34}{'Simulation A':>20}{'Simulation B':>20}")
    print(f"{'-' * 34}{'-' * 20:>20}{'-' * 20:>20}")
    for name, a, b in rows:
        print(f"{name:<34}{str(a):>20}{str(b):>20}")

    def higher(name, a, b, unit="", decimals=2):
        if abs(a - b) < 1e-9:
            print(f"  {name:<28}: equal ({a:.{decimals}f}{unit})")
        else:
            winner = "A" if a > b else "B"
            print(f"  {name:<28}: Simulation {winner} ({max(a, b):.{decimals}f}{unit}"
                  f" vs {min(a, b):.{decimals}f}{unit})")

    print("\nWhich simulation is higher?")
    higher("Distance from start", stats_a['displacement'], stats_b['displacement'], " cells")
    higher("Grid coverage", stats_a['coverage'], stats_b['coverage'], "%", 1)
    higher("Spread around centroid", stats_a['spread'], stats_b['spread'], " cells")
    higher("Steps stayed in place", stats_a['avg_stayed'], stats_b['avg_stayed'], " per agent", 1)


def save_csv(models):
    """Task 1 and Task 3: record every path and every final position."""
    with open("TA1_agent_paths.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["simulation", "agent", "step", "x", "y"])
        for label, model in models:
            for agent in model.agents:
                for t, (x, y) in enumerate(agent.path):
                    writer.writerow([label, agent.id, t, x, y])
    with open("TA1_final_positions.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["simulation", "agent", "start_x", "start_y",
                         "final_x", "final_y", "moves", "stayed"])
        for label, model in models:
            for agent in model.agents:
                writer.writerow([label, agent.id, agent.path[0][0], agent.path[0][1],
                                 agent.position[0], agent.position[1],
                                 agent.moves, agent.stayed])


# ---------------------------------------------------------------------------
# Visualization helpers
# ---------------------------------------------------------------------------
def style_grid_axes(ax, width, height):
    """Agents sit in the middle of each cell; grid lines mark cell borders."""
    ax.set_xlim(-0.5, width - 0.5)
    ax.set_ylim(-0.5, height - 0.5)
    ax.set_aspect('equal')
    step = 1 if width <= 20 else 5
    ax.set_xticks(range(0, width, step))
    ax.set_yticks(range(0, height, step))
    ax.set_xticks([i - 0.5 for i in range(width + 1)], minor=True)
    ax.set_yticks([i - 0.5 for i in range(height + 1)], minor=True)
    ax.grid(True, which='minor', color='lightgray', linewidth=0.6)
    ax.tick_params(which='minor', length=0)
    ax.set_xlabel("x")
    ax.set_ylabel("y")


def agent_color(i, total):
    # Strong colors for small groups, 20 colors (repeating) for bigger groups
    return plt.cm.tab10(i) if total <= 10 else plt.cm.tab20(i % 20)


class PathPanel:
    """Draws one simulation (trails + current positions) on one axes."""

    def __init__(self, ax, model, label):
        self.ax, self.model, self.label = ax, model, label
        width, height = model.p.grid_size
        style_grid_axes(ax, width, height)
        total = len(model.agents)
        self.colors = [agent_color(i, total) for i in range(total)]
        self.paths = [agent.path for agent in model.agents]
        # Small fixed offset per agent so trails that share a grid line stay visible
        self.offsets = [((i % 7) - 3) * 0.05 for i in range(total)]
        starts = [path[0] for path in self.paths]
        ax.scatter([s[0] for s in starts], [s[1] for s in starts], marker='s', s=60,
```

```
                facecolors='none', edgecolors=self.colors, linewidths=1.2, zorder=2)
        self.lines = [ax.plot([], [], '-', color=c, linewidth=1.6, alpha=0.75, zorder=1)[0]
                      for c in self.colors]
        self.dots = ax.scatter([s[0] for s in starts], [s[1] for s in starts], s=140,
                               color=self.colors, edgecolors='black', linewidths=0.6, zorder=3)
        self.title = ax.set_title("")

    def draw(self, frame):
        t = min(frame, self.model.p.steps)          # shorter run stays on its last step
        for line, path, off in zip(self.lines, self.paths, self.offsets):
            line.set_data([pos[0] + off for pos in path[:t + 1]],
                          [pos[1] + off for pos in path[:t + 1]])
        self.dots.set_offsets([path[t] for path in self.paths])
        self.title.set_text(f"Simulation {self.label}: {describe(self.model.p)}\n"
                            f"step {t} of {self.model.p.steps}")
        return self.lines + [self.dots, self.title]


def legend_handles():
    return [Line2D([], [], marker='s', color='gray', markerfacecolor='none', linestyle='',
                   label='Start'),
            Line2D([], [], color='gray', linewidth=1.6, label='Path'),
            Line2D([], [], marker='o', color='gray', markeredgecolor='black', linestyle='',
                   markersize=9, label='Current position')]


def draw_heatmap(ax, values, width, height, title, cmap, annotate):
    image = ax.imshow(values, origin='lower', cmap=cmap, aspect='equal', vmin=0,
                      extent=(-0.5, width - 0.5, -0.5, height - 0.5))
    style_grid_axes(ax, width, height)
    ax.set_title(title, fontsize=10)
    colorbar = plt.colorbar(image, ax=ax, fraction=0.046, pad=0.04)
    colorbar.locator = MaxNLocator(integer=True)     # counts are whole numbers
    colorbar.update_ticks()
    if annotate and width <= 15:
        top = max(max(row) for row in values)
        for y in range(height):
            for x in range(width):
                if values[y][x]:
                    color = 'white' if values[y][x] > top / 2 else 'black'
                    ax.text(x, y, str(values[y][x]), ha='center', va='center',
                            fontsize=8, color=color)


def analysis_figure(models):
    """Task 3 figure: final positions and how often each cell was visited."""
    fig, axes = plt.subplots(len(models), 2, figsize=(11, 5 * len(models)))
    for row, (label, model) in enumerate(models):
        width, height = model.p.grid_size
        finals = [[0] * width for _ in range(height)]
        visits = [[0] * width for _ in range(height)]
        for agent in model.agents:
            finals[agent.position[1]][agent.position[0]] += 1
            for x, y in agent.path:
                visits[y][x] += 1
        draw_heatmap(axes[row][0], finals, width, height,
                     f"Simulation {label}: final positions (agents per cell)", 'Blues', True)
        half_x, half_y = math.ceil(width / 2), math.ceil(height / 2)
        axes[row][0].axvline(half_x - 0.5, color='red', linestyle='--', linewidth=1)
        axes[row][0].axhline(half_y - 0.5, color='red', linestyle='--', linewidth=1)
        draw_heatmap(axes[row][1], visits, width, height,
                     f"Simulation {label}: visits per cell over the whole run", 'YlOrRd', False)
    fig.suptitle("Final Position Distribution (red dashed lines = quadrant borders)")
    fig.tight_layout()
    return fig


def spread_figure(models):
    """Task 4 figure: how the agents spread out step by step."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))
    for label, model in models:
        steps = range(len(model.displacement_history))
        ax1.plot(steps, model.displacement_history, label=f"{label}: {describe(model.p)}")
        ax2.plot(steps, model.coverage_history, label=f"{label}: {describe(model.p)}")
    ax1.set_title("Average distance from starting cell")
    ax1.set_xlabel("Step")
    ax1.set_ylabel("Distance (cells)")
    ax2.set_title("Grid coverage (cells visited at least once)")
```

```
    ax2.set_xlabel("Step")
    ax2.set_ylabel("Coverage (%)")
    ax2.set_ylim(0, 105)
    for ax in (ax1, ax2):
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=8, loc='lower right')
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------------------
# Main program
# ---------------------------------------------------------------------------
def main():
    print(LINE)
    print("RANDOM WALK SIMULATION USING AGENTPY - MODIFIED CODE (TA1)")
    print(LINE)

    # -------- INPUT --------
    print("[INPUT] Enter the parameters of the two simulations to compare.")
    seed = ask_seed()
    params_a = get_scenario("A", seed)
    params_b = get_scenario("B", seed)

    # -------- PROCESS --------
    print_header("[PROCESS] Running the simulations")
    models = []
    for params in (params_a, params_b):
        model = RandomWalkModel(params)
        model.run(display=False)          # AgentPy runs setup, then step() 'steps' times
        print(f"Simulation {params['label']}: {describe(params)} -> {model.t} steps completed")
        models.append((params['label'], model))
    print(f"Seed used for both simulations: {seed}")
    print("(Same seed = same starting cells when the agent count and grid size match.)")
    (_, model_a), (_, model_b) = models
    stats_a, stats_b = analyze(model_a), analyze(model_b)

    # -------- OUTPUT --------
    print_header("[OUTPUT] TASK 1 - AGENT PATHS")
    print_paths(model_a, "A")
    print_paths(model_b, "B")

    print_header("[OUTPUT] TASK 2 - MOVEMENT RULE CHECK")
    print_rule_check(model_a, stats_a, "A")
    print_rule_check(model_b, stats_b, "B")

    print_header("[OUTPUT] TASK 3 - FINAL POSITIONS AND DISTRIBUTION")
    print_final_positions(model_a, stats_a, "A")
    print_final_positions(model_b, stats_b, "B")

    print_header("[OUTPUT] TASK 4 - SCENARIO COMPARISON")
    print_comparison(model_a, stats_a, model_b, stats_b)

    # -------- Save records and figures --------
    save_csv(models)

    # Figure 1: path animation (A and B side by side)
    fig1, axes = plt.subplots(1, 2, figsize=(13, 6.5))
    panels = [PathPanel(ax, model, label) for ax, (label, model) in zip(axes, models)]
    fig1.legend(handles=legend_handles(), loc='lower center', ncol=3)
    last_frame = max(model_a.p.steps, model_b.p.steps)
    for panel in panels:
        panel.draw(last_frame)
    fig1.tight_layout(rect=(0, 0.06, 1, 1))
    # The final frame shows the complete path of every agent
    fig1.savefig("TA1_fig1_agent_paths.png", dpi=150, bbox_inches='tight')

    fig2 = analysis_figure(models)
    fig2.savefig("TA1_fig2_final_positions.png", dpi=150, bbox_inches='tight')
    fig3 = spread_figure(models)
    fig3.savefig("TA1_fig3_spread_over_time.png", dpi=150, bbox_inches='tight')

    print_header("[OUTPUT] FILES SAVED")
    print("  TA1_agent_paths.csv            - every position of every agent, step by step")
    print("  TA1_final_positions.csv        - start and final position of every agent")
    print("  TA1_fig1_agent_paths.png       - complete paths of all agents (Task 1)")
    print("  TA1_fig2_final_positions.png   - final position distribution (Task 3)")
    print("  TA1_fig3_spread_over_time.png  - spread of the agents over time (Task 4)")
```

```
    # -------- Interactive Animation --------
    def update(frame):
        artists = []
        for panel in panels:
            artists += panel.draw(frame)
        return artists

    print("\nShowing the animation and charts. Close all figure windows to end the program.")
    ani = animation.FuncAnimation(fig1, update, frames=last_frame + 1, interval=250,
                                  init_func=lambda: update(0), blit=False, repeat=False)
    plt.show()
    print("Program finished.")
    return ani


if __name__ == "__main__":
    main()
```

Sample Run 1: Comparing Parameters (Tasks 1, 3 and 4)

Inputs: seed 2026. Simulation A uses 5 agents, a 10x10 grid, 20 steps and Rule 1 (random walk). Simulation B uses 20 agents, a 10x10 grid, 50 steps and Rule 1.

Console Output - Sample Run 1

```
================================================================================
RANDOM WALK SIMULATION USING AGENTPY - MODIFIED CODE (TA1)
================================================================================
[INPUT] Enter the parameters of the two simulations to compare.
Enter random seed (press Enter for a random seed): 2026

--- Simulation A ---
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
Movement rules: [1] Random walk  [2] Avoid each other  [3] Preferred direction
Choose movement rule (1-3): 1

--- Simulation B ---
Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50
Movement rules: [1] Random walk  [2] Avoid each other  [3] Preferred direction
Choose movement rule (1-3): 1

================================================================================
[PROCESS] Running the simulations
================================================================================
Simulation A: Random walk, 5 agents, 10x10 grid, 20 steps -> 20 steps completed
Simulation B: Random walk, 20 agents, 10x10 grid, 50 steps -> 50 steps completed
Seed used for both simulations: 2026
(Same seed = same starting cells when the agent count and grid size match.)

================================================================================
[OUTPUT] TASK 1 - AGENT PATHS
================================================================================

Simulation A: Random walk, 5 agents, 10x10 grid, 20 steps
Agent  Start    Moves Stayed Visited  Path (first 5 positions)
-----  -------- ----- ------ -------  ----------------------------------------
    1  (6,5)       20      0      10  (6,5)->(6,4)->(6,3)->(7,3)->(7,2) ...
    2  (8,2)       20      0      14  (8,2)->(7,2)->(6,2)->(6,3)->(6,2) ...
    3  (1,3)       20      0      18  (1,3)->(2,3)->(2,4)->(2,3)->(3,3) ...
    4  (2,8)       18      2      15  (2,8)->(3,8)->(3,9)->(3,9)->(3,9) ...
    5  (7,6)       19      1      11  (7,6)->(8,6)->(8,5)->(8,6)->(8,5) ...
  Moves = steps that changed cell, Stayed = steps spent in place, Visited = distinct cells

Simulation B: Random walk, 20 agents, 10x10 grid, 50 steps
Agent  Start    Moves Stayed Visited  Path (first 5 positions)
-----  -------- ----- ------ -------  ----------------------------------------
    1  (6,5)       41      9      20  (6,5)->(6,4)->(6,5)->(5,5)->(5,4) ...
    2  (8,2)       42      8      21  (8,2)->(8,3)->(8,4)->(8,5)->(9,5) ...
    3  (1,3)       48      2      20  (1,3)->(1,4)->(2,4)->(3,4)->(4,4) ...
    4  (2,8)       50      0      26  (2,8)->(2,7)->(2,6)->(1,6)->(0,6) ...
    5  (7,6)       49      1      21  (7,6)->(7,7)->(7,8)->(8,8)->(8,7) ...
    6  (7,9)       40     10      27  (7,9)->(7,9)->(7,9)->(8,9)->(8,8) ...
```

```
    7  (7,1)       33     17      11  (7,1)->(8,1)->(7,1)->(7,2)->(8,2) ...
    8  (5,3)       50      0      21  (5,3)->(5,2)->(5,1)->(4,1)->(4,2) ...
    9  (7,3)       46      4      23  (7,3)->(7,4)->(7,5)->(8,5)->(8,6) ...
   10  (7,0)       37     13      22  (7,0)->(7,0)->(7,0)->(8,0)->(8,0) ...
   11  (9,3)       46      4      19  (9,3)->(9,3)->(9,2)->(9,2)->(9,1) ...
   12  (9,9)       50      0      23  (9,9)->(8,9)->(8,8)->(8,7)->(7,7) ...
   13  (9,8)       46      4      26  (9,8)->(9,7)->(8,7)->(7,7)->(7,6) ...
   14  (6,2)       44      6      17  (6,2)->(6,1)->(6,0)->(6,1)->(7,1) ...
   15  (9,6)       37     13      17  (9,6)->(9,6)->(9,5)->(9,4)->(9,4) ...
   16  (7,5)       50      0      21  (7,5)->(8,5)->(8,6)->(7,6)->(7,7) ...
   17  (5,6)       47      3      31  (5,6)->(5,7)->(4,7)->(5,7)->(5,8) ...
   18  (3,0)       35     15      16  (3,0)->(3,0)->(4,0)->(4,0)->(4,1) ...
   19  (0,0)       43      7      32  (0,0)->(0,0)->(0,0)->(0,0)->(1,0) ...
   20  (7,8)       43      7      25  (7,8)->(7,9)->(7,8)->(8,8)->(9,8) ...
  Moves = steps that changed cell, Stayed = steps spent in place, Visited = distinct cells

================================================================================
[OUTPUT] TASK 2 - MOVEMENT RULE CHECK
================================================================================

Simulation A: Random walk, 5 agents, 10x10 grid, 20 steps
  Rule              : picks Up/Down/Left/Right with equal chance (25% each)
  Directions chosen : Right 23.0% | Left 19.0% | Up 25.0% | Down 33.0%  (n = 100)
  Moves made        : Right 23.7% | Left 19.6% | Up 23.7% | Down 33.0%  (n = 97)
  Stayed put        : 3 time(s) (blocked by: edge of the grid)
  Most agents sharing one cell at any step: 2

Simulation B: Random walk, 20 agents, 10x10 grid, 50 steps
  Rule              : picks Up/Down/Left/Right with equal chance (25% each)
  Directions chosen : Right 26.6% | Left 23.5% | Up 24.0% | Down 25.9%  (n = 1000)
  Moves made        : Right 24.5% | Left 25.8% | Up 24.6% | Down 25.1%  (n = 877)
  Stayed put        : 123 time(s) (blocked by: edge of the grid)
  Most agents sharing one cell at any step: 3

================================================================================
[OUTPUT] TASK 3 - FINAL POSITIONS AND DISTRIBUTION
================================================================================

Simulation A: Random walk, 5 agents, 10x10 grid, 20 steps
Final positions (agent: x,y):
    1:(4,5)       2:(2,2)       3:(6,2)       4:(8,2)       5:(8,4)
Final position map (number = agents in the cell, . = empty, top row is y = 9):
  y= 9 |  .  .  .  .  .  .  .  .  .  .
  y= 8 |  .  .  .  .  .  .  .  .  .  .
  y= 7 |  .  .  .  .  .  .  .  .  .  .
  y= 6 |  .  .  .  .  .  .  .  .  .  .
  y= 5 |  .  .  .  .  1  .  .  .  .  .
  y= 4 |  .  .  .  .  .  .  .  .  1  .
  y= 3 |  .  .  .  .  .  .  .  .  .  .
  y= 2 |  .  .  1  .  .  .  1  .  1  .
  y= 1 |  .  .  .  .  .  .  .  .  .  .
  y= 0 |  .  .  .  .  .  .  .  .  .  .
       +------------------------------
      x =  0  1  2  3  4  5  6  7  8  9
  Centroid (average x, y)          : (5.60, 3.00)
  Std. deviation of x, y           : (2.33, 1.26)
  Avg distance from the centroid   : 2.51 cells
  Avg distance from grid center    : 3.00 cells
  Quadrants  Q1 upper-right: 0 | Q2 upper-left: 1 | Q3 lower-left: 1 | Q4 lower-right: 3
  Agents on the border cells       : 0 of 5 (0.0%)
  Occupied cells at the end        : 5 of 100 (max 1 agent(s) in one cell)
  Grid coverage (cells visited)    : 46.0%
  Boundary check                   : PASSED - every recorded position is inside the grid

Simulation B: Random walk, 20 agents, 10x10 grid, 50 steps
Final positions (agent: x,y):
    1:(4,8)       2:(7,3)       3:(5,1)       4:(7,9)       5:(8,4)       6:(9,1)
    7:(8,1)       8:(0,6)       9:(5,7)      10:(0,0)      11:(6,4)      12:(6,8)
   13:(8,3)      14:(9,1)      15:(5,9)      16:(4,6)      17:(3,1)      18:(9,1)
   19:(6,3)      20:(1,7)
Final position map (number = agents in the cell, . = empty, top row is y = 9):
  y= 9 |  .  .  .  .  .  1  .  1  .  .
  y= 8 |  .  .  .  .  1  .  1  .  .  .
  y= 7 |  .  1  .  .  .  1  .  .  .  .
  y= 6 |  1  .  .  .  1  .  .  .  .  .
  y= 5 |  .  .  .  .  .  .  .  .  .  .
  y= 4 |  .  .  .  .  .  .  1  .  1  .
  y= 3 |  .  .  .  .  .  .  1  1  1  .
```

```
  y= 2 |  .  .  .  .  .  .  .  .  .  .
  y= 1 |  .  .  .  1  .  1  .  .  1  3
  y= 0 |  1  .  .  .  .  .  .  .  .  .
       +------------------------------
          x =  0  1  2  3  4  5  6  7  8  9
  Centroid (average x, y)          : (5.50, 4.15)
  Std. deviation of x, y           : (2.77, 3.00)
  Avg distance from the centroid   : 3.78 cells
  Avg distance from grid center    : 4.00 cells
  Quadrants  Q1 upper-right: 4 | Q2 upper-left: 4 | Q3 lower-left: 2 | Q4 lower-right: 10
  Agents on the border cells       : 7 of 20 (35.0%)
  Occupied cells at the end        : 18 of 100 (max 3 agent(s) in one cell)
  Grid coverage (cells visited)    : 99.0%
  Boundary check                   : PASSED - every recorded position is inside the grid

================================================================================
[OUTPUT] TASK 4 - SCENARIO COMPARISON
================================================================================
Metric                                    Simulation A        Simulation B
--------------------------------------------------------------------------------
Number of agents                                     5                  20
Grid size                                        10x10               10x10
Number of steps                                     20                  50
Movement rule                              Random walk         Random walk
Agent density (agents per cell)                   0.05                0.20
Avg distance from start                           4.76                4.52
Avg moves per agent                               19.4                43.9
Avg steps stayed in place                          0.6                 6.2
Grid coverage                                    46.0%               99.0%
Avg distance from centroid                        2.51                3.78
Agents on border cells                            0.0%               35.0%
Occupied cells at the end                            5                  18
Max agents in one cell (end)                         1                   3

Which simulation is higher?
  Distance from start         : Simulation A (4.76 cells vs 4.52 cells)
  Grid coverage               : Simulation B (99.0% vs 46.0%)
  Spread around centroid      : Simulation B (3.78 cells vs 2.51 cells)
  Steps stayed in place       : Simulation B (6.2 per agent vs 0.6 per agent)

================================================================================
[OUTPUT] FILES SAVED
================================================================================
  TA1_agent_paths.csv            - every position of every agent, step by step
  TA1_final_positions.csv        - start and final position of every agent
  TA1_fig1_agent_paths.png       - complete paths of all agents (Task 1)
  TA1_fig2_final_positions.png   - final position distribution (Task 3)
  TA1_fig3_spread_over_time.png  - spread of the agents over time (Task 4)

Showing the animation and charts. Close all figure windows to end the program.
Program finished.
```

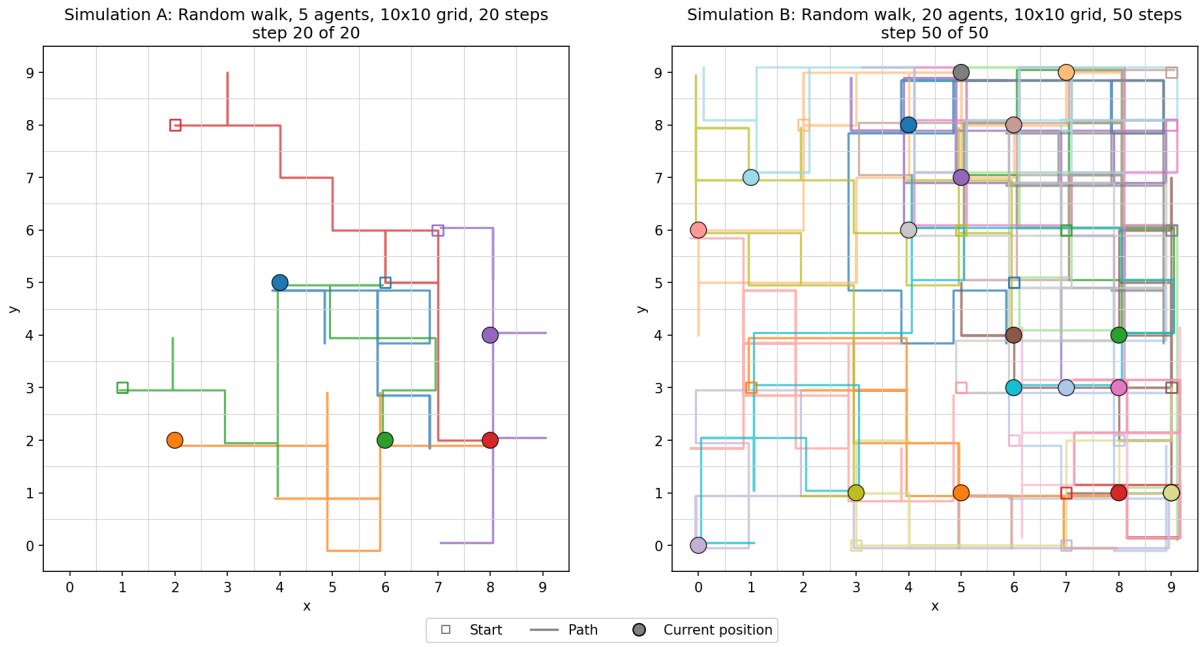


Figure 3. Run 1 complete agent paths (final frame of the animation).

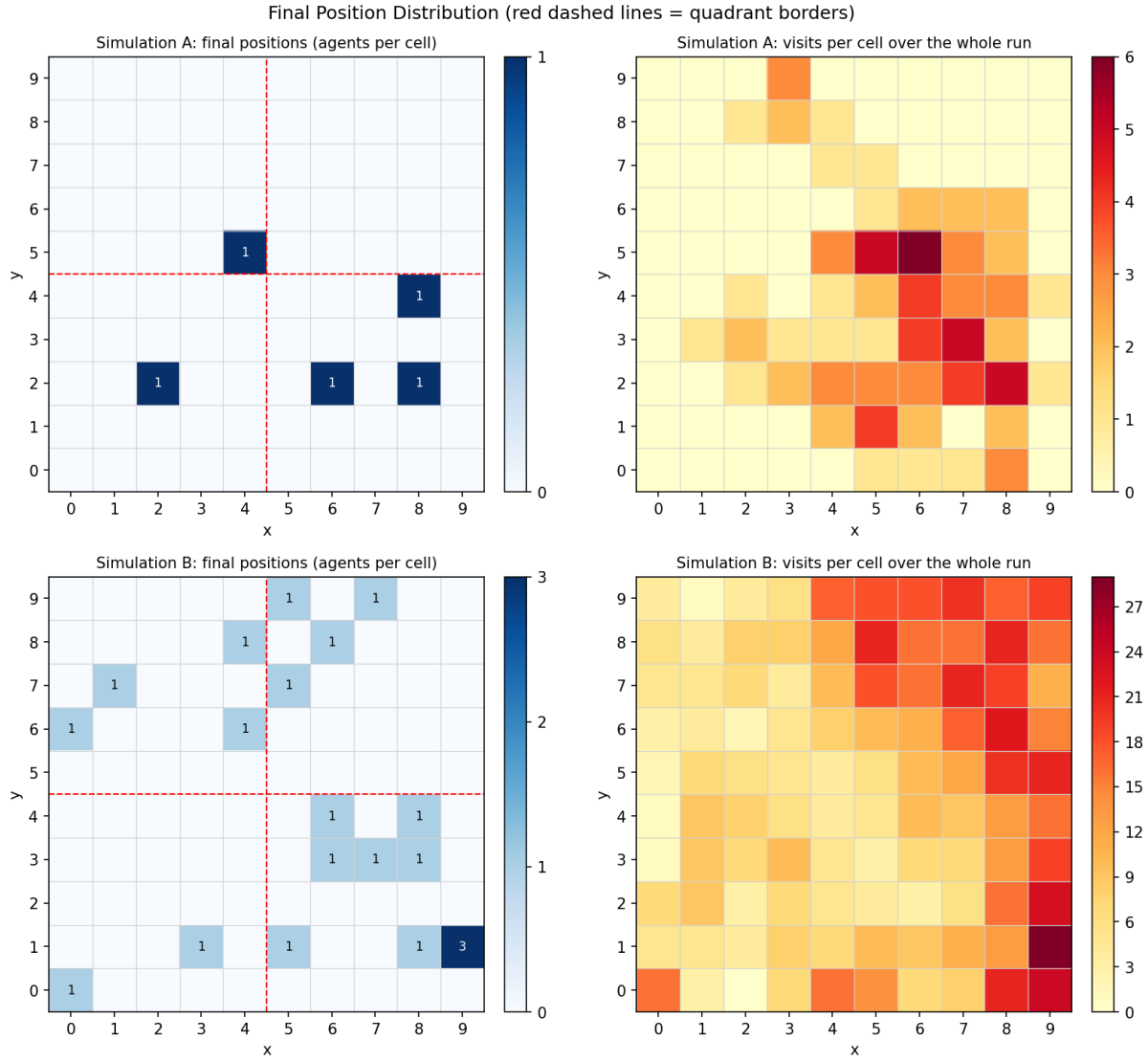


Figure 4. Run 1 final position distribution and cell visits.

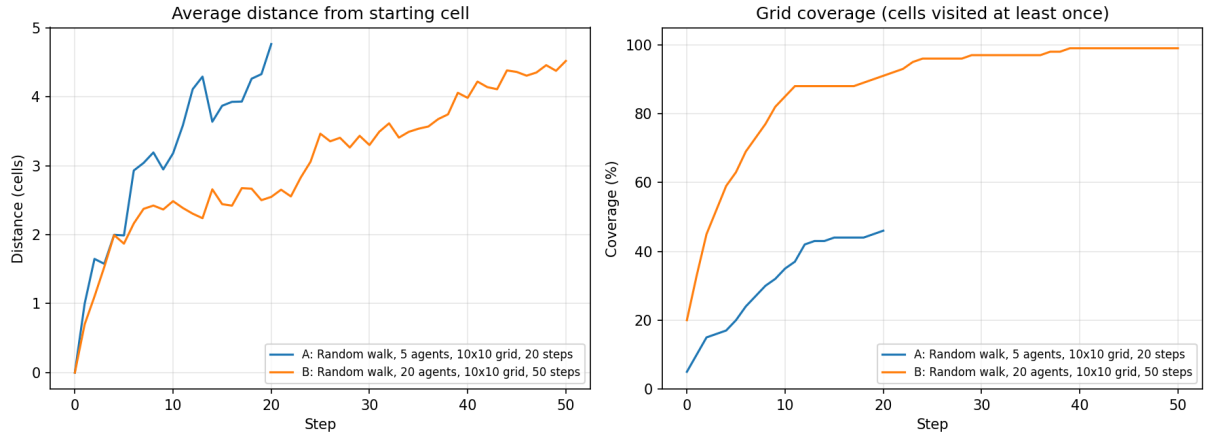


Figure 5. Run 1 spread of the agents over time.

Run 1 observation. Both simulations used the plain random walk, so the differences come from the parameters. Simulation B, with four times the agents and 2.5 times the steps, visited 99% of the grid compared with 46% for A, and its agents ended more spread out (average distance from the centroid of 3.78 vs 2.51 cells). Longer runs also mean more wall hits: B's agents spent 6.2 steps each stuck against an edge compared with 0.6 for A. A's agents actually ended slightly farther from their starting cells on average (4.76 vs 4.52 cells). With only five agents that average swings a lot by chance, and Figure 5 shows A's curve climbing much faster than B's did over the same first 20 steps. B's final positions were not perfectly even either (10 of 20 agents ended in the lower-right quadrant), which is normal for a small random sample.

Sample Run 2: Changing the Movement Rule (Task 2)

Inputs: seed 2026. Both simulations use 20 agents, a 10x10 grid and 50 steps. Simulation A uses Rule 2 (avoid each other) and Simulation B uses Rule 3 with Right as the preferred direction. The seed, agent count and grid are the same as Simulation B of Run 1, so all three rules start from exactly the same cells.

Console Output - Sample Run 2

```
================================================================================
RANDOM WALK SIMULATION USING AGENTPY - MODIFIED CODE (TA1)
================================================================================
[INPUT] Enter the parameters of the two simulations to compare.
Enter random seed (press Enter for a random seed): 2026

--- Simulation A ---
Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50
Movement rules: [1] Random walk  [2] Avoid each other  [3] Preferred direction
Choose movement rule (1-3): 2

--- Simulation B ---
Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50
Movement rules: [1] Random walk  [2] Avoid each other  [3] Preferred direction
Choose movement rule (1-3): 3
Enter preferred direction (U/D/L/R): R

================================================================================
[PROCESS] Running the simulations
================================================================================
Simulation A: Avoid each other, 20 agents, 10x10 grid, 50 steps -> 50 steps completed
Simulation B: Preferred direction (Right), 20 agents, 10x10 grid, 50 steps -> 50 steps completed
Seed used for both simulations: 2026
(Same seed = same starting cells when the agent count and grid size match.)

================================================================================
[OUTPUT] TASK 1 - AGENT PATHS
================================================================================

Simulation A: Avoid each other, 20 agents, 10x10 grid, 50 steps
Agent  Start    Moves Stayed Visited  Path (first 5 positions)
-----  -------- ----- ------ -------  ----------------------------------------
    1  (6,5)       50      0      24  (6,5)->(6,4)->(6,5)->(6,6)->(6,5) ...
    2  (8,2)       50      0      23  (8,2)->(8,3)->(8,2)->(8,3)->(8,4) ...
    3  (1,3)       50      0      16  (1,3)->(1,2)->(1,1)->(0,1)->(0,2) ...
    4  (2,8)       50      0      20  (2,8)->(2,7)->(1,7)->(0,7)->(1,7) ...
    5  (7,6)       49      1      17  (7,6)->(7,7)->(7,6)->(8,6)->(7,6) ...
    6  (7,9)       50      0      13  (7,9)->(7,8)->(7,7)->(7,8)->(8,8) ...
    7  (7,1)       50      0      22  (7,1)->(8,1)->(8,0)->(7,0)->(8,0) ...
    8  (5,3)       50      0      25  (5,3)->(5,2)->(5,1)->(5,0)->(6,0) ...
    9  (7,3)       49      1      22  (7,3)->(7,4)->(8,4)->(9,4)->(9,3) ...
   10  (7,0)       50      0      23  (7,0)->(7,1)->(7,2)->(7,1)->(6,1) ...
   11  (9,3)       50      0      12  (9,3)->(9,4)->(9,3)->(9,2)->(9,1) ...
   12  (9,9)       50      0      31  (9,9)->(8,9)->(8,8)->(9,8)->(9,9) ...
   13  (9,8)       50      0      14  (9,8)->(9,9)->(9,8)->(9,7)->(9,6) ...
   14  (6,2)       50      0      28  (6,2)->(6,3)->(7,3)->(6,3)->(5,3) ...
   15  (9,6)       50      0      25  (9,6)->(8,6)->(8,7)->(7,7)->(6,7) ...
   16  (7,5)       50      0      25  (7,5)->(8,5)->(9,5)->(9,6)->(9,5) ...
   17  (5,6)       50      0      26  (5,6)->(5,7)->(5,6)->(4,6)->(4,5) ...
   18  (3,0)       50      0      28  (3,0)->(2,0)->(2,1)->(2,2)->(3,2) ...
   19  (0,0)       50      0      14  (0,0)->(0,1)->(0,0)->(1,0)->(2,0) ...
```

```
   20  (7,8)       50      0      26  (7,8)->(6,8)->(6,7)->(5,7)->(4,7) ...
  Moves = steps that changed cell, Stayed = steps spent in place, Visited = distinct cells

Simulation B: Preferred direction (Right), 20 agents, 10x10 grid, 50 steps
Agent  Start    Moves Stayed Visited  Path (first 5 positions)
-----  -------- ----- ------ -------  ----------------------------------------
    1  (6,5)       25     25      15  (6,5)->(6,6)->(7,6)->(7,5)->(8,5) ...
    2  (8,2)       34     16      11  (8,2)->(7,2)->(8,2)->(8,3)->(7,3) ...
    3  (1,3)       32     18      17  (1,3)->(1,2)->(1,1)->(2,1)->(3,1) ...
    4  (2,8)       36     14      17  (2,8)->(1,8)->(2,8)->(3,8)->(2,8) ...
    5  (7,6)       30     20      13  (7,6)->(8,6)->(9,6)->(9,6)->(9,6) ...
    6  (7,9)       26     24      13  (7,9)->(8,9)->(8,8)->(9,8)->(8,8) ...
    7  (7,1)       38     12      13  (7,1)->(8,1)->(7,1)->(7,0)->(7,1) ...
    8  (5,3)       37     13      15  (5,3)->(5,4)->(5,3)->(5,2)->(5,1) ...
    9  (7,3)       35     15      14  (7,3)->(8,3)->(8,4)->(8,5)->(9,5) ...
   10  (7,0)       34     16      17  (7,0)->(6,0)->(6,1)->(5,1)->(6,1) ...
   11  (9,3)       34     16      15  (9,3)->(9,2)->(9,3)->(9,4)->(9,4) ...
   12  (9,9)       33     17      11  (9,9)->(8,9)->(8,8)->(9,8)->(9,9) ...
   13  (9,8)       34     16      16  (9,8)->(9,9)->(9,9)->(9,8)->(9,7) ...
   14  (6,2)       42      8      22  (6,2)->(6,3)->(6,2)->(7,2)->(8,2) ...
   15  (9,6)       31     19      12  (9,6)->(9,5)->(8,5)->(8,4)->(7,4) ...
   16  (7,5)       36     14      17  (7,5)->(7,6)->(6,6)->(6,5)->(5,5) ...
   17  (5,6)       41      9      16  (5,6)->(6,6)->(7,6)->(7,7)->(7,6) ...
   18  (3,0)       36     14      15  (3,0)->(3,0)->(3,1)->(4,1)->(5,1) ...
   19  (0,0)       40     10      21  (0,0)->(0,1)->(0,2)->(1,2)->(2,2) ...
   20  (7,8)       21     29      10  (7,8)->(7,9)->(7,9)->(8,9)->(7,9) ...
  Moves = steps that changed cell, Stayed = steps spent in place, Visited = distinct cells

================================================================================
[OUTPUT] TASK 2 - MOVEMENT RULE CHECK
================================================================================

Simulation A: Avoid each other, 20 agents, 10x10 grid, 50 steps
  Rule              : picks randomly among the empty neighbouring cells inside the grid
  Directions chosen : Right 22.7% | Left 25.2% | Up 26.2% | Down 26.0%  (n = 998)
  Moves made        : Right 22.7% | Left 25.2% | Up 26.2% | Down 26.0%  (n = 998)
  Stayed put        : 2 time(s) (blocked by: no empty neighbouring cell)
  Most agents sharing one cell at any step: 1

Simulation B: Preferred direction (Right), 20 agents, 10x10 grid, 50 steps
  Rule              : moves Right 50% of the time, other directions 16.7% each
  Directions chosen : Right 50.1% | Left 15.5% | Up 16.0% | Down 18.4%  (n = 1000)
  Moves made        : Right 30.8% | Left 23.0% | Up 22.2% | Down 24.0%  (n = 675)
  Stayed put        : 325 time(s) (blocked by: edge of the grid)
  Most agents sharing one cell at any step: 5

================================================================================
[OUTPUT] TASK 3 - FINAL POSITIONS AND DISTRIBUTION
================================================================================

Simulation A: Avoid each other, 20 agents, 10x10 grid, 50 steps
Final positions (agent: x,y):
    1:(0,9)       2:(4,6)       3:(4,0)       4:(0,2)       5:(7,9)       6:(9,5)
    7:(8,2)       8:(7,3)       9:(6,5)      10:(7,0)      11:(8,0)      12:(1,3)
   13:(8,5)      14:(3,7)      15:(9,8)      16:(2,4)      17:(5,8)      18:(3,6)
   19:(1,1)      20:(5,6)
Final position map (number = agents in the cell, . = empty, top row is y = 9):
  y= 9 |  1  .  .  .  .  .  .  1  .  .
  y= 8 |  .  .  .  .  .  1  .  .  .  1
  y= 7 |  .  .  .  1  .  .  .  .  .  .
  y= 6 |  .  .  .  1  1  1  .  .  .  .
  y= 5 |  .  .  .  .  .  .  1  .  1  1
  y= 4 |  .  .  1  .  .  .  .  .  .  .
  y= 3 |  .  1  .  .  .  .  .  1  .  .
  y= 2 |  1  .  .  .  .  .  .  .  1  .
  y= 1 |  .  1  .  .  .  .  .  .  .  .
  y= 0 |  .  .  .  .  1  .  .  1  1  .
       +------------------------------
         x =  0  1  2  3  4  5  6  7  8  9
  Centroid (average x, y)          : (4.85, 4.45)
  Std. deviation of x, y           : (2.94, 2.91)
  Avg distance from the centroid   : 3.87 cells
  Avg distance from grid center    : 3.88 cells
  Quadrants  Q1 upper-right: 7 | Q2 upper-left: 4 | Q3 lower-left: 5 | Q4 lower-right: 4
  Agents on the border cells       : 8 of 20 (40.0%)
  Occupied cells at the end        : 20 of 100 (max 1 agent(s) in one cell)
  Grid coverage (cells visited)    : 100.0%
  Boundary check                   : PASSED - every recorded position is inside the grid
```

```
Simulation B: Preferred direction (Right), 20 agents, 10x10 grid, 50 steps
Final positions (agent: x,y):
    1:(9,9)       2:(9,5)       3:(8,0)       4:(9,7)       5:(9,2)       6:(9,1)
    7:(9,3)       8:(7,2)       9:(9,4)      10:(8,5)      11:(9,3)      12:(9,6)
   13:(8,3)      14:(9,7)      15:(9,1)      16:(9,3)      17:(8,8)      18:(9,0)
   19:(9,1)      20:(9,5)
Final position map (number = agents in the cell, . = empty, top row is y = 9):
  y= 9 |  .  .  .  .  .  .  .  .  .  1
  y= 8 |  .  .  .  .  .  .  .  .  1  .
  y= 7 |  .  .  .  .  .  .  .  .  .  2
  y= 6 |  .  .  .  .  .  .  .  .  .  1
  y= 5 |  .  .  .  .  .  .  .  .  1  2
  y= 4 |  .  .  .  .  .  .  .  .  .  1
  y= 3 |  .  .  .  .  .  .  .  .  1  3
  y= 2 |  .  .  .  .  .  .  .  1  .  1
  y= 1 |  .  .  .  .  .  .  .  .  .  3
  y= 0 |  .  .  .  .  .  .  .  .  1  1
       +------------------------------
     x =  0  1  2  3  4  5  6  7  8  9
  Centroid (average x, y)          : (8.70, 3.75)
  Std. deviation of x, y           : (0.56, 2.61)
  Avg distance from the centroid   : 2.31 cells
  Avg distance from grid center    : 4.97 cells
  Quadrants  Q1 upper-right: 8 | Q2 upper-left: 0 | Q3 lower-left: 0 | Q4 lower-right: 12
  Agents on the border cells       : 16 of 20 (80.0%)
  Occupied cells at the end        : 14 of 100 (max 3 agent(s) in one cell)
  Grid coverage (cells visited)    : 68.0%
  Boundary check                   : PASSED - every recorded position is inside the grid

================================================================================
[OUTPUT] TASK 4 - SCENARIO COMPARISON
================================================================================
Metric                                   Simulation A        Simulation B
--------------------------------------------------------------------------------
Number of agents                                   20                  20
Grid size                                       10x10               10x10
Number of steps                                    50                  50
Movement rule                       Avoid each other   Preferred (Right)
Agent density (agents per cell)                  0.20                0.20
Avg distance from start                          3.90                4.60
Avg moves per agent                              49.9                33.8
Avg steps stayed in place                         0.1                16.2
Grid coverage                                   100.0%              68.0%
Avg distance from centroid                       3.87                2.31
Agents on border cells                           40.0%               80.0%
Occupied cells at the end                          20                  14
Max agents in one cell (end)                        1                   3

Which simulation is higher?
  Distance from start         : Simulation B (4.60 cells vs 3.90 cells)
  Grid coverage               : Simulation A (100.0% vs 68.0%)
  Spread around centroid      : Simulation A (3.87 cells vs 2.31 cells)
  Steps stayed in place       : Simulation B (16.2 per agent vs 0.1 per agent)

================================================================================
[OUTPUT] FILES SAVED
================================================================================
  TA1_agent_paths.csv            - every position of every agent, step by step
  TA1_final_positions.csv        - start and final position of every agent
  TA1_fig1_agent_paths.png       - complete paths of all agents (Task 1)
  TA1_fig2_final_positions.png   - final position distribution (Task 3)
  TA1_fig3_spread_over_time.png  - spread of the agents over time (Task 4)

Showing the animation and charts. Close all figure windows to end the program.
Program finished.
```

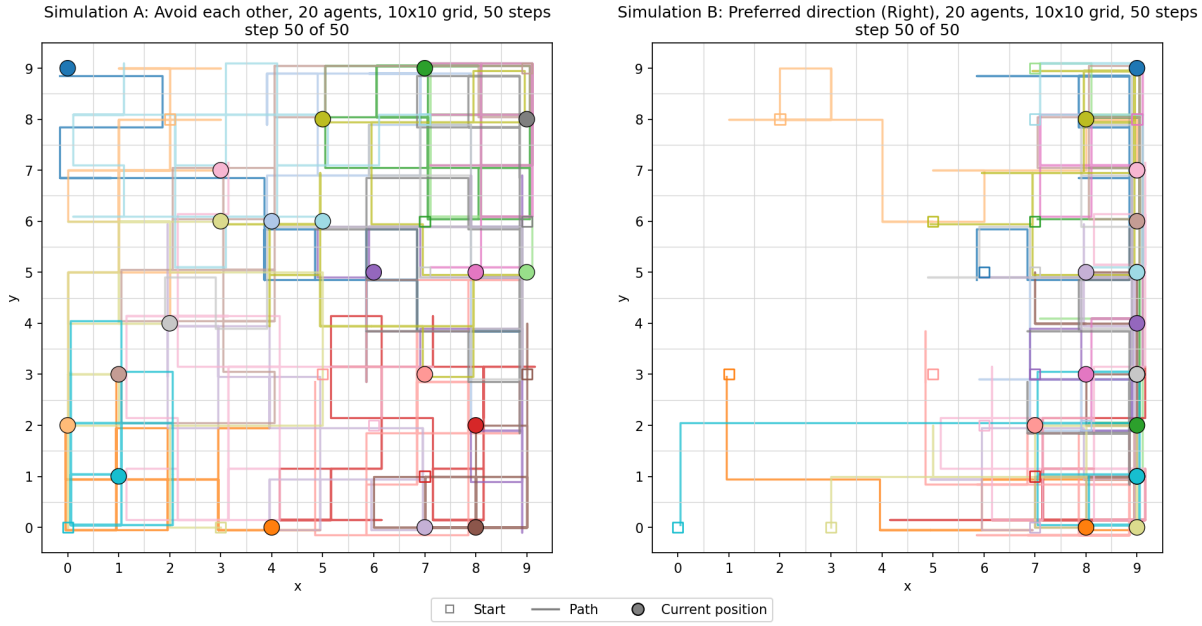


Figure 6. Run 2 complete agent paths: avoid each other (A) vs preferred direction Right (B).

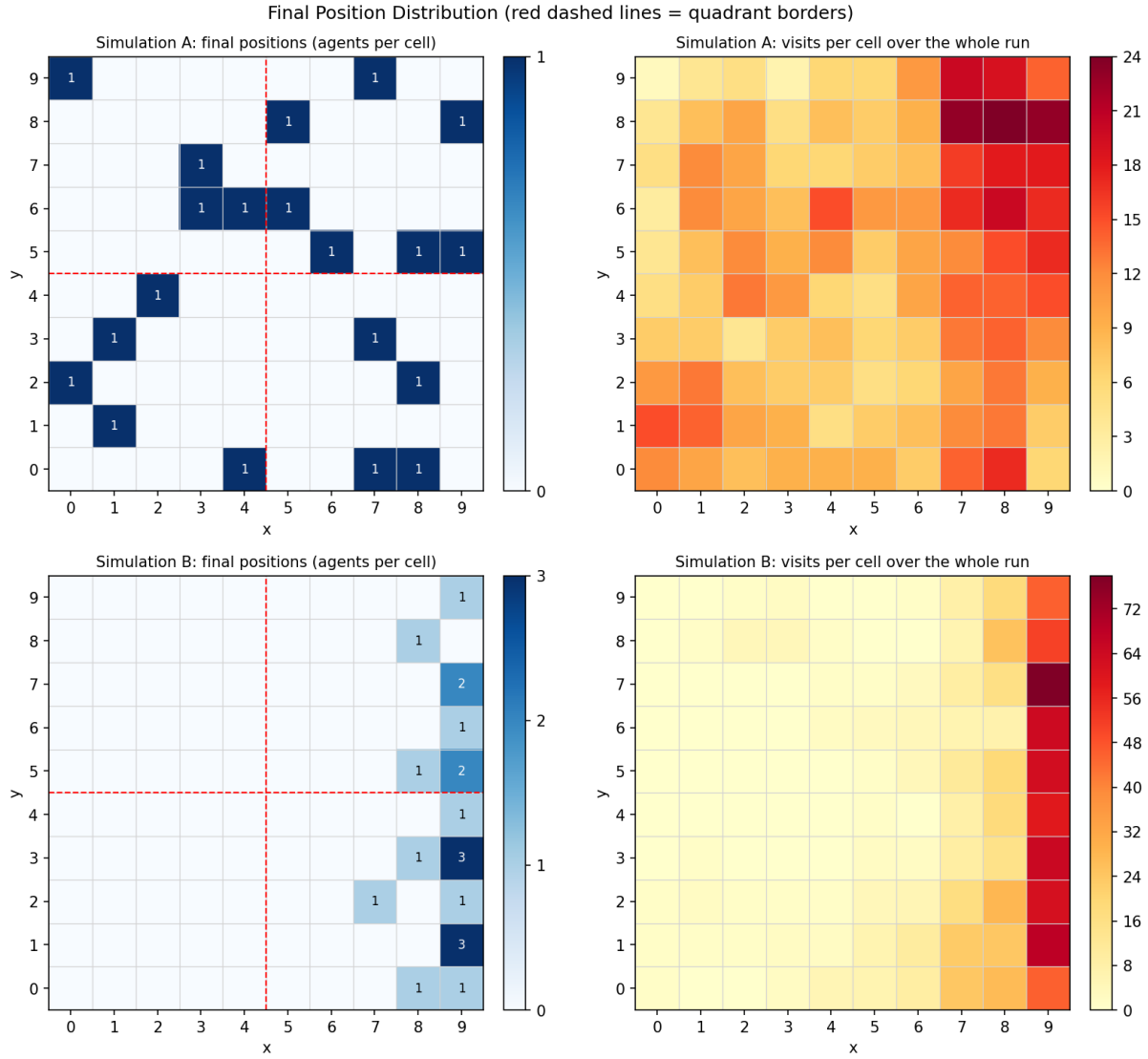


Figure 7. Run 2 final position distribution and cell visits.

Run 2 observation. With the avoid rule, no two agents ever shared a cell (the most agents in one cell at any step was 1, compared with 3 for the plain random walk in Run 1), agents were almost never stuck (2 blocked steps in total) and they spread across the whole grid, visiting 100% of the cells. The preferred-direction rule produced a very different pattern. Agents chose Right 50.1% of the time, so they drifted to the right wall and piled up there. By the end, all 20 agents were in the right half, 80% were on edge cells and the average x position was 8.70 on a 0 to 9 scale. Once at the wall their right moves were cancelled, which is why they stayed in place 16.2 steps per agent and actual moves to the right dropped to 30.8%. Coverage also fell to 68% because the left side was rarely visited.

Sample Run 3: Crowded Grid (supports Question 1)

Inputs: seed 2026. Both simulations use 90 agents on a 10x10 grid (0.9 agents per cell) for 50 steps. Simulation A uses Rule 1 (random walk) and Simulation B uses Rule 2 (avoid each other).

Console Output - Sample Run 3

```
================================================================================
RANDOM WALK SIMULATION USING AGENTPY - MODIFIED CODE (TA1)
================================================================================
```

```
[INPUT] Enter the parameters of the two simulations to compare.
Enter random seed (press Enter for a random seed): 2026

--- Simulation A ---
Enter number of agents: 90
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50
Movement rules: [1] Random walk  [2] Avoid each other  [3] Preferred direction
Choose movement rule (1-3): 1

--- Simulation B ---
Enter number of agents: 90
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50
Movement rules: [1] Random walk  [2] Avoid each other  [3] Preferred direction
Choose movement rule (1-3): 2

================================================================================
[PROCESS] Running the simulations
================================================================================
Simulation A: Random walk, 90 agents, 10x10 grid, 50 steps -> 50 steps completed
Simulation B: Avoid each other, 90 agents, 10x10 grid, 50 steps -> 50 steps completed
Seed used for both simulations: 2026
(Same seed = same starting cells when the agent count and grid size match.)

================================================================================
[OUTPUT] TASK 1 - AGENT PATHS
================================================================================

Simulation A: Random walk, 90 agents, 10x10 grid, 50 steps
Agent  Start    Moves Stayed Visited  Path (first 5 positions)
-----  -------- ----- ------ -------  ----------------------------------------
    1  (6,5)       49      1      23  (6,5)->(7,5)->(7,6)->(6,6)->(6,7) ...
    2  (8,2)       40     10      14  (8,2)->(9,2)->(8,2)->(8,1)->(8,0) ...
    3  (1,3)       48      2      19  (1,3)->(0,3)->(0,3)->(1,3)->(1,4) ...
    4  (2,8)       45      5      27  (2,8)->(2,9)->(2,9)->(2,9)->(2,9) ...
    5  (7,6)       48      2      29  (7,6)->(8,6)->(8,5)->(7,5)->(6,5) ...
    6  (7,9)       50      0      26  (7,9)->(6,9)->(6,8)->(6,7)->(5,7) ...
    7  (7,1)       39     11      22  (7,1)->(7,0)->(7,1)->(8,1)->(8,0) ...
    8  (5,3)       47      3      26  (5,3)->(6,3)->(6,4)->(6,5)->(6,4) ...
    9  (7,3)       38     12      19  (7,3)->(7,2)->(8,2)->(9,2)->(9,2) ...
   10  (7,0)       50      0      34  (7,0)->(8,0)->(8,1)->(7,1)->(7,0) ...
   11  (6,2)       40     10      21  (6,2)->(7,2)->(7,1)->(7,2)->(8,2) ...
   12  (7,5)       50      0      14  (7,5)->(6,5)->(6,6)->(6,7)->(7,7) ...
   13  (5,6)       44      6      14  (5,6)->(4,6)->(3,6)->(3,5)->(2,5) ...
   14  (3,0)       48      2      36  (3,0)->(2,0)->(2,1)->(2,2)->(2,3) ...
   15  (0,0)       48      2      22  (0,0)->(0,1)->(0,1)->(0,2)->(1,2) ...
   16  (7,8)       46      4      23  (7,8)->(6,8)->(7,8)->(6,8)->(7,8) ...
   17  (1,0)       45      5      17  (1,0)->(0,0)->(1,0)->(2,0)->(1,0) ...
   18  (1,4)       43      7      21  (1,4)->(0,4)->(1,4)->(0,4)->(0,4) ...
   19  (3,6)       40     10      13  (3,6)->(2,6)->(1,6)->(1,7)->(2,7) ...
   20  (1,2)       49      1      28  (1,2)->(0,2)->(1,2)->(2,2)->(3,2) ...
   21  (5,7)       48      2      28  (5,7)->(5,8)->(5,7)->(5,8)->(4,8) ...
   22  (0,1)       48      2      19  (0,1)->(0,0)->(0,1)->(1,1)->(1,0) ...
   23  (8,9)       42      8      25  (8,9)->(9,9)->(9,8)->(9,7)->(9,8) ...
   24  (4,0)       47      3      26  (4,0)->(5,0)->(4,0)->(5,0)->(5,0) ...
   25  (2,6)       49      1      22  (2,6)->(2,5)->(3,5)->(4,5)->(4,6) ...
  ... 65 more agents (all paths are in TA1_agent_paths.csv)
  Moves = steps that changed cell, Stayed = steps spent in place, Visited = distinct cells

Simulation B: Avoid each other, 90 agents, 10x10 grid, 50 steps
Agent  Start    Moves Stayed Visited  Path (first 5 positions)
-----  -------- ----- ------ -------  ----------------------------------------
    1  (6,5)       20     30      13  (6,5)->(6,5)->(6,5)->(7,5)->(7,6) ...
    2  (8,2)       20     30       6  (8,2)->(8,2)->(8,2)->(9,2)->(9,2) ...
    3  (1,3)       17     33      11  (1,3)->(1,3)->(1,2)->(1,2)->(1,2) ...
    4  (2,8)       21     29      11  (2,8)->(2,8)->(2,9)->(2,8)->(1,8) ...
    5  (7,6)       20     30      13  (7,6)->(7,5)->(8,5)->(8,5)->(7,5) ...
    6  (7,9)       18     32       8  (7,9)->(7,9)->(7,9)->(7,9)->(7,9) ...
    7  (7,1)       19     31      10  (7,1)->(7,1)->(7,0)->(7,1)->(7,1) ...
    8  (5,3)       19     31      10  (5,3)->(5,3)->(5,3)->(6,3)->(5,3) ...
    9  (7,3)       23     27      10  (7,3)->(7,3)->(7,3)->(7,3)->(7,2) ...
   10  (7,0)       18     32       9  (7,0)->(7,0)->(8,0)->(7,0)->(7,0) ...
   11  (6,2)       19     31      13  (6,2)->(6,2)->(6,2)->(5,2)->(5,2) ...
   12  (7,5)       15     35       7  (7,5)->(8,5)->(9,5)->(9,5)->(9,5) ...
   13  (5,6)       19     31      14  (5,6)->(5,6)->(6,6)->(5,6)->(5,5) ...
   14  (3,0)        7     43       5  (3,0)->(2,0)->(2,0)->(2,0)->(2,0) ...
   15  (0,0)       18     32      13  (0,0)->(0,1)->(0,1)->(0,1)->(0,1) ...
```

```
   16  (7,8)       16     34      14  (7,8)->(7,8)->(7,8)->(7,8)->(7,8) ...
   17  (1,0)       11     39      11  (1,0)->(0,0)->(0,0)->(0,0)->(0,0) ...
   18  (1,4)       22     28      18  (1,4)->(1,4)->(1,4)->(1,3)->(1,3) ...
   19  (3,6)       20     30      13  (3,6)->(3,6)->(3,6)->(3,6)->(2,6) ...
   20  (1,2)       14     36      10  (1,2)->(1,1)->(1,1)->(1,1)->(1,1) ...
   21  (5,7)       18     32      13  (5,7)->(5,7)->(4,7)->(4,7)->(4,6) ...
   22  (0,1)       15     35      10  (0,1)->(0,2)->(0,2)->(0,2)->(0,2) ...
   23  (8,9)       16     34      11  (8,9)->(8,9)->(8,9)->(8,9)->(8,9) ...
   24  (4,0)       16     34      12  (4,0)->(3,0)->(3,0)->(3,0)->(3,0) ...
   25  (2,6)       17     33      13  (2,6)->(2,6)->(2,6)->(2,6)->(2,5) ...
  ... 65 more agents (all paths are in TA1_agent_paths.csv)
  Moves = steps that changed cell, Stayed = steps spent in place, Visited = distinct cells

================================================================================
[OUTPUT] TASK 2 - MOVEMENT RULE CHECK
================================================================================

Simulation A: Random walk, 90 agents, 10x10 grid, 50 steps
  Rule              : picks Up/Down/Left/Right with equal chance (25% each)
  Directions chosen : Right 24.7% | Left 25.3% | Up 25.4% | Down 24.5%  (n = 4500)
  Moves made        : Right 24.7% | Left 25.4% | Up 25.3% | Down 24.6%  (n = 4084)
  Stayed put        : 416 time(s) (blocked by: edge of the grid)
  Most agents sharing one cell at any step: 7

Simulation B: Avoid each other, 90 agents, 10x10 grid, 50 steps
  Rule              : picks randomly among the empty neighbouring cells inside the grid
  Directions chosen : Right 24.5% | Left 25.8% | Up 25.2% | Down 24.4%  (n = 1487)
  Moves made        : Right 24.5% | Left 25.8% | Up 25.2% | Down 24.4%  (n = 1487)
  Stayed put        : 3013 time(s) (blocked by: no empty neighbouring cell)
  Most agents sharing one cell at any step: 1

================================================================================
[OUTPUT] TASK 3 - FINAL POSITIONS AND DISTRIBUTION
================================================================================

Simulation A: Random walk, 90 agents, 10x10 grid, 50 steps
Final positions (agent: x,y):
    1:(2,6)       2:(9,3)       3:(0,4)       4:(3,4)       5:(3,4)       6:(5,5)
    7:(1,2)       8:(8,7)       9:(3,1)      10:(5,6)      11:(7,9)      12:(6,8)
   13:(1,4)      14:(6,7)      15:(0,4)      16:(9,4)      17:(6,2)      18:(1,9)
   19:(1,6)      20:(9,3)      21:(3,1)      22:(3,2)      23:(0,3)      24:(4,5)
   25:(3,6)      26:(6,7)      27:(3,7)      28:(0,2)      29:(2,7)      30:(5,8)
   31:(9,6)      32:(4,9)      33:(4,6)      34:(1,6)      35:(1,0)      36:(5,0)
   37:(0,8)      38:(5,1)      39:(1,7)      40:(4,4)      41:(8,5)      42:(6,1)
   43:(0,4)      44:(1,4)      45:(7,7)      46:(1,1)      47:(3,5)      48:(0,1)
   49:(4,1)      50:(0,8)      51:(4,9)      52:(2,7)      53:(1,7)      54:(1,6)
   55:(7,4)      56:(1,7)      57:(3,8)      58:(5,9)      59:(1,5)      60:(4,9)
   61:(8,1)      62:(9,3)      63:(7,7)      64:(4,4)      65:(5,7)      66:(2,7)
   67:(5,4)      68:(5,5)      69:(9,9)      70:(9,5)      71:(1,9)      72:(9,5)
   73:(4,0)      74:(9,3)      75:(9,5)      76:(8,3)      77:(0,2)      78:(9,8)
   79:(4,1)      80:(7,7)      81:(6,7)      82:(9,2)      83:(2,2)      84:(8,2)
   85:(8,9)      86:(7,4)      87:(1,3)      88:(2,4)      89:(9,5)      90:(7,0)
Final position map (number = agents in the cell, . = empty, top row is y = 9):
  y= 9 |  .  2  .  .  3  1  .  1  1  1
  y= 8 |  2  .  .  1  .  1  1  .  .  1
  y= 7 |  .  3  3  1  .  1  3  3  1  .
  y= 6 |  .  3  1  1  1  1  .  .  .  1
  y= 5 |  .  1  .  1  1  2  .  .  1  4
  y= 4 |  3  2  1  2  2  1  .  2  .  1
  y= 3 |  1  1  .  .  .  .  .  .  1  4
  y= 2 |  2  1  1  1  .  .  1  .  1  1
  y= 1 |  1  1  .  2  2  1  1  .  1  .
  y= 0 |  .  1  .  .  1  1  .  1  .  .
       +------------------------------
          x =  0  1  2  3  4  5  6  7  8  9
  Centroid (average x, y)          : (4.32, 4.82)
  Std. deviation of x, y           : (3.02, 2.63)
  Avg distance from the centroid   : 3.76 cells
  Avg distance from grid center    : 3.79 cells
  Quadrants  Q1 upper-right: 24 | Q2 upper-left: 24 | Q3 lower-left: 25 | Q4 lower-right: 17
  Agents on the border cells       : 34 of 90 (37.8%)
  Occupied cells at the end        : 60 of 100 (max 4 agent(s) in one cell)
  Grid coverage (cells visited)    : 100.0%
  Boundary check                   : PASSED - every recorded position is inside the grid

Simulation B: Avoid each other, 90 agents, 10x10 grid, 50 steps
Final positions (agent: x,y):
    1:(3,8)       2:(9,1)       3:(1,2)       4:(0,7)       5:(5,0)       6:(6,8)
```

```
    7:(6,1)       8:(8,3)       9:(8,1)      10:(5,2)      11:(9,4)      12:(9,8)
   13:(7,9)      14:(2,2)      15:(5,3)      16:(4,5)      17:(5,1)      18:(9,6)
   19:(7,4)      20:(3,4)      21:(9,9)      22:(1,5)      23:(4,7)      24:(7,3)
   25:(4,3)      26:(2,0)      27:(4,0)      28:(2,7)      29:(6,9)      30:(7,7)
   31:(8,9)      32:(0,3)      33:(2,5)      34:(0,2)      35:(2,8)      36:(7,2)
   37:(5,6)      38:(9,7)      39:(5,9)      40:(7,1)      41:(8,4)      42:(4,8)
   43:(6,6)      44:(0,9)      45:(3,1)      46:(4,6)      47:(4,4)      48:(0,4)
   49:(0,0)      50:(1,8)      51:(0,8)      52:(3,6)      53:(5,5)      54:(1,7)
   55:(6,4)      56:(1,1)      57:(2,6)      58:(1,4)      59:(8,2)      60:(3,9)
   61:(6,5)      62:(2,9)      63:(3,0)      64:(3,7)      65:(6,7)      66:(3,5)
   67:(4,1)      68:(5,8)      69:(9,5)      70:(1,9)      71:(1,6)      72:(7,0)
   73:(0,5)      74:(2,4)      75:(7,6)      76:(2,1)      77:(1,3)      78:(7,5)
   79:(6,0)      80:(8,8)      81:(3,2)      82:(5,4)      83:(2,3)      84:(8,7)
   85:(6,3)      86:(8,0)      87:(4,9)      88:(9,3)      89:(5,7)      90:(4,2)
Final position map (number = agents in the cell, . = empty, top row is y = 9):
  y= 9 |  1  1  1  1  1  1  1  1  1  1
  y= 8 |  1  1  1  1  1  1  1  .  1  1
  y= 7 |  1  1  1  1  1  1  1  1  1  1
  y= 6 |  .  1  1  1  1  1  1  1  .  1
  y= 5 |  1  1  1  1  1  1  1  1  .  1
  y= 4 |  1  1  1  1  1  1  1  1  1  1
  y= 3 |  1  1  1  .  1  1  1  1  1  1
  y= 2 |  1  1  1  1  1  1  .  1  1  .
  y= 1 |  .  1  1  1  1  1  1  1  1  1
  y= 0 |  1  .  1  1  1  1  1  1  1  .
       +------------------------------
         x =  0  1  2  3  4  5  6  7  8  9
  Centroid (average x, y)          : (4.43, 4.63)
  Std. deviation of x, y           : (2.78, 2.87)
  Avg distance from the centroid   : 3.74 cells
  Avg distance from grid center    : 3.74 cells
  Quadrants  Q1 upper-right: 22 | Q2 upper-left: 24 | Q3 lower-left: 22 | Q4 lower-right: 22
  Agents on the border cells       : 31 of 90 (34.4%)
  Occupied cells at the end        : 90 of 100 (max 1 agent(s) in one cell)
  Grid coverage (cells visited)    : 100.0%
  Boundary check                   : PASSED - every recorded position is inside the grid

================================================================================
[OUTPUT] TASK 4 - SCENARIO COMPARISON
================================================================================
Metric                                    Simulation A        Simulation B
--------------------------------------------------------------------------------
Number of agents                                    90                  90
Grid size                                        10x10               10x10
Number of steps                                     50                  50
Movement rule                              Random walk    Avoid each other
Agent density (agents per cell)                   0.90                0.90
Avg distance from start                           4.29                2.94
Avg moves per agent                               45.4                16.5
Avg steps stayed in place                          4.6                33.5
Grid coverage                                   100.0%              100.0%
Avg distance from centroid                        3.76                3.74
Agents on border cells                           37.8%               34.4%
Occupied cells at the end                           60                  90
Max agents in one cell (end)                         4                   1

Which simulation is higher?
  Distance from start         : Simulation A (4.29 cells vs 2.94 cells)
  Grid coverage               : equal (100.0%)
  Spread around centroid      : Simulation A (3.76 cells vs 3.74 cells)
  Steps stayed in place       : Simulation B (33.5 per agent vs 4.6 per agent)

================================================================================
[OUTPUT] FILES SAVED
================================================================================
  TA1_agent_paths.csv            - every position of every agent, step by step
  TA1_final_positions.csv        - start and final position of every agent
  TA1_fig1_agent_paths.png       - complete paths of all agents (Task 1)
  TA1_fig2_final_positions.png   - final position distribution (Task 3)
  TA1_fig3_spread_over_time.png  - spread of the agents over time (Task 4)

Showing the animation and charts. Close all figure windows to end the program.
Program finished.
```

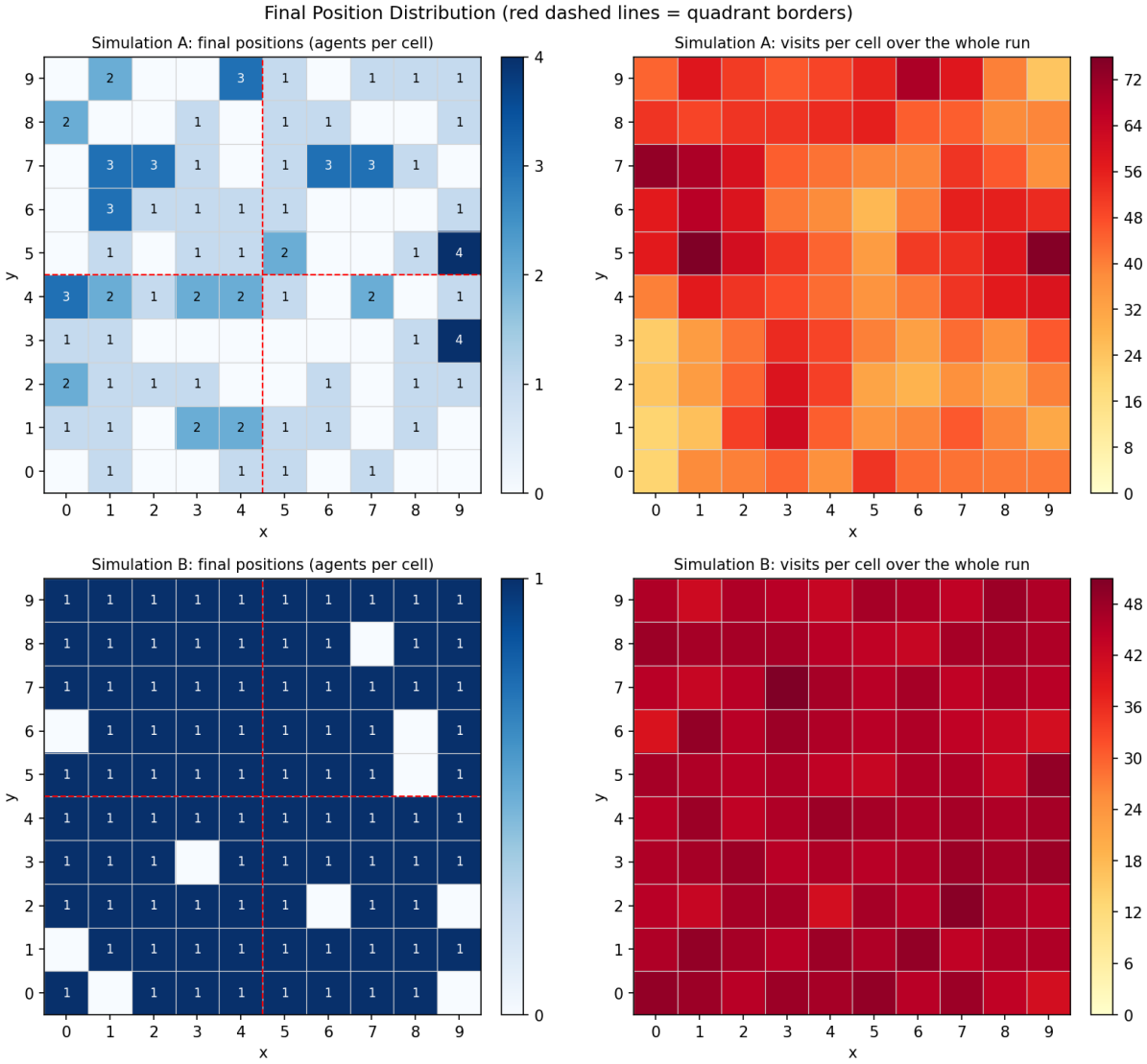


Figure 8. Run 3 final position distribution with 90 agents on a 10x10 grid.

Run 3 observation. Crowding changes the behavior a lot. Under the random walk, agents stacked up so only 60 of the 100 cells were occupied at the end, with up to 4 agents in one cell (7 at one point during the run). Under the avoid rule every agent kept its own cell, but there was so little free space that agents were blocked for 33.5 of the 50 steps on average and ended only 2.94 cells from their start, compared with 4.29 cells for the random walk.

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
5.

Answer the Questions ⇩

1. What happens if the number of agents is very high compared to the grid size?

The grid fills up fast and the agents lose room to move freely. With the original random rule, agents simply stack on the same cells. In Run 3 (90 agents on a 10x10 grid) only 60 cells were occupied at the end and up to 7 agents shared one cell during the run, so the plot shows far fewer dots than there are agents. If the agents have to avoid each other, crowding turns into near gridlock: they were stuck for 33.5 of 50 steps on average and moved much less overall. Once there are more agents than cells, sharing cells can't be avoided at all.

2. How does increasing the number of steps affect the spread of agents?

More steps means more spread, but the growth slows down. In Run 1 Simulation B, the average distance from the starting cell went from about 2.5 cells at step 20 to 4.52 cells at step 50, while grid coverage jumped to about 85% in the first 10 steps and then only crept up to 99% (Figure 5). This fits random-walk theory, where the typical distance grows roughly with the square root of the number of steps, so doubling the steps does not double the spread. In a bounded grid the walls eventually cap the spread, and over a long run the agents end up spread evenly across the whole grid. Comparing Simulation A (20 steps) with B (50 steps) shows the same trend, with coverage of 46% vs 99% and spread around the centroid of 2.51 vs 3.78 cells, although B also had more agents.

3. Do the agents ever move outside the grid? Why or why not?

No. Every move is clamped with max(0, min(grid_size - 1, x + dx)), so a step that would cross an edge is cancelled and the agent stays on its boundary cell for that turn. Nothing wraps around to the opposite side either, because the code clamps instead of using modulo and the grid is set with torus=False. The avoid rule only considers neighbouring cells that are inside the grid. The modified program checks every recorded position, and the boundary check passed in all six simulations. The "Stayed put" counts, such as 123 blocked steps in Run 1 Simulation B, are exactly the moves that were cancelled at the edge.

4. How can this simulation be used to model real-life phenomena (e.g., diffusion, crowd movement)?

Each agent can stand for a particle, a person or an animal, and the grid is the space they move in. With the plain random walk, agents gradually spread out to fill the whole space, which is how diffusion works (ink spreading in water or a smell filling a room); the coverage curve in Figure 5 rises quickly and then levels off as the space fills. The avoid rule works like people keeping personal space in a crowd, and Run 3 shows how a packed area quickly turns into gridlock. The preferred direction works like a crowd heading toward an exit or animals moving toward food, and Run 2 shows the bottleneck that forms at the target wall. The same model can be extended to the spread of disease (agents in the same cell count as contacts), animal foraging, or robot swarms searching an area.